Лабораторная работа № 3

Тема: Проектирование реляционной базы данных интернет-магазина. Первичные и внешние ключи, ограничения целостности.

Цель работы: расширить базу shop_db до пяти связанных таблиц, закрепить проектирование атрибутов и типов данных, настроить PRIMARY KEY, FOREIGN KEY, NOT NULL, UNIQUE, CHECK и проверить оформление тестового заказа.

Основной инструмент: pgAdmin 4. Таблицы и ограничения создаются преимущественно через графический интерфейс; Query Tool используется для проверки данных и ограничений.

1. Исходные условия

- Продолжайте существующую базу shop_db проекта shop-api; не создавайте новую базу вместо неё.
- Предполагается, что categories и products уже существуют.
- В целевой структуре products.category_id ссылается на categories.id.
- Новые таблицы: users, orders, order_items.
- Не удаляйте имеющиеся товары и категории, не сбрасывайте базу данных.

Если products ещё хранит категорию текстом в поле category, сначала перенесите названия в categories, заполните category_id по соответствующим категориям и проверьте каждую связь. Только после переноса можно отказаться от текстового поля category. Не удаляйте данные без резервной копии.

Используемые типы и ограничения соответствуют лекции: INTEGER для идентификаторов и сумм в целых тенге, VARCHAR для строк, BOOLEAN для признаков, TIMESTAMPTZ для событий; INTEGER GENERATED ALWAYS AS IDENTITY для автоматически создаваемых идентификаторов.

2. План целевой базы данных

Таблица	Назначение	Связи
categories	Категории товаров	products.category_id → categories.id
products	Товары и цены	order_items.product_id → products.id
users	Покупатели	orders.user_id → users.id
orders	Заказы покупателей	order_items.order_id → orders.id
order_items	Состав заказов	Ссылки на orders и products

Всего должно получиться пять таблиц и четыре внешних ключа. Каждую связь создайте по направлению от дочерней таблицы (где расположен FK) к родительской (на чьё id ссылается FK).

3. Подготовка существующих таблиц

- Откройте pgAdmin → Servers → PostgreSQL → Databases → shop_db → Schemas → public → Tables.
- Откройте Properties → Columns таблицы categories: убедитесь, что id имеет PRIMARY KEY, name имеет тип character varying(100), NOT NULL и UNIQUE.

- Проверьте products: id, name, price, category_id, in_stock; для price должно быть CHECK (price > 0).
- Откройте Properties → Constraints → Foreign Key в products: проверьте ссылку Local column category_id → public.categories.id.
- Сохраните скриншоты структуры обеих таблиц.

Если ограничения отличаются от целевой модели, приведите их к требованиям. Перед включением NOT NULL и FK убедитесь, что существующие строки соответствуют ограничениям.

4. Создание таблицы users через pgAdmin

В дереве базы данных откройте public → Tables → правой кнопкой → Create → Table. В General укажите Name: users. Затем во вкладке Columns добавьте поля по таблице ниже.

Поле	Тип	Требование
id	integer	PRIMARY KEY, GENERATED ALWAYS AS IDENTITY
name	character varying(100)	NOT NULL
email	character varying(100)	NOT NULL, UNIQUE
password_hash	character varying(255)	NOT NULL
created_at	timestamp with time zone	NOT NULL, DEFAULT now()

- Для id включите Primary key и настройте Identity через свойства столбца (в версии pgAdmin параметры могут располагаться в разделе Identity).
- Для email добавьте UNIQUE: Constraints → Unique → «+» → Columns → email.
- Для created_at в Default укажите now().
- Сохраните таблицу и убедитесь, что она появилась в списке Tables.

Пароли открытым текстом хранить нельзя. В этой лабораторной работа с аутентификацией не реализуется. Поле password_hash предназначено для результата надёжного хеширования пароля, который будет получать приложение. Не вводите реальные пароли покупателей.

5. Создание таблицы orders через pgAdmin

Создайте таблицу orders. Используйте параметры столбцов:

Поле	Тип	Требование
id	integer	PRIMARY KEY, IDENTITY
user_id	integer	NOT NULL, FOREIGN KEY → users.id
order_date	timestamp with time zone	NOT NULL, DEFAULT now()
status	character varying(20)	NOT NULL, DEFAULT 'new'
total_amount	integer	NOT NULL, CHECK (total_amount >= 0)

- Во вкладке Constraints → Foreign Key нажмите «+».

- Укажите имя fk_orders_users.
- На вкладке Columns задайте Local column: user_id; References: public.users; Referencing: id.
- Нажмите кнопку добавления соответствия столбцов внутри вкладки Columns и затем сохраните ограничение.
- Через Constraints → Check добавьте условие total_amount >= 0.
- Сохраните таблицу.

status=new — пример статуса учебного заказа. total_amount — сумма заказа в целых тенге; CHECK запрещает отрицательные значения, но сам по себе не вычисляет сумму позиций.

6. Создание таблицы order_items через pgAdmin

Создайте таблицу order_items — каждая строка будет представлять конкретный товар в оформленном заказе.

Поле	Тип	Требование
id	integer	PRIMARY KEY, IDENTITY
order_id	integer	NOT NULL, FOREIGN KEY → orders.id
product_id	integer	NOT NULL, FOREIGN KEY → products.id
quantity	integer	NOT NULL, CHECK (quantity > 0)
price	integer	NOT NULL, CHECK (price > 0)

- Создайте внешний ключ fk_order_items_orders: order_id → public.orders.id.
- Создайте внешний ключ fk_order_items_products: product_id → public.products.id.
- Добавьте CHECK (quantity > 0) и CHECK (price > 0).
- Убедитесь, что обе связи отображаются в Constraints → Foreign Key.
- Сохраните таблицу.

Поле order_items.price хранит цену единицы товара на момент оформления заказа; оно не обязано автоматически изменяться при последующем изменении products.price.

7. Проверка четырёх связей

Дочерняя таблица	Local column	Родительская таблица	Referenced column
products	category_id	categories	id
orders	user_id	users	id
order_items	order_id	orders	id
order_items	product_id	products	id

Для проверки откройте у каждой дочерней таблицы Properties → Constraints → Foreign Key. Сделайте скриншоты; отдельно проверьте типы столбцов обеих сторон каждой связи.

8. Наполнение учебными данными

Через View/Edit Data → All Rows добавьте как минимум одного тестового покупателя. Поле id оставьте для автоматической генерации. Для password_hash используйте только заранее подготовленную тестовую строку, не реальный пароль; это учебные данные, не сценарий авторизации.

Для примера заказа требуются два товара: «Ноутбук» по 300000 ₸ и «Мышь» по 8000 ₸. Если они уже существуют, используйте их реальные id. Если отсутствуют, добавьте их в products, выбрав существующую категорию category_id.

Проверить реальные идентификаторы покупателей и товаров можно в Query Tool:

```
SELECT id, name, email FROM users ORDER BY id;
SELECT id, name, price, category_id FROM products ORDER BY id;
```

Не используйте демонстрационные id 5, 10, 15, 101 из лекции как обязательные: база каждой группы может назначить другие идентификаторы.

9. Оформление тестового заказа

- Выберите одного существующего пользователя и запишите его users.id.
- Проверьте реальные идентификаторы ноутбука и мыши и цены в products.
- В orders создайте строку с user_id выбранного пользователя, status = new и total_amount = 316000; id и order_date сформируйте по умолчанию.
- Найдите присвоенный номер заказа orders.id.
- Создайте в order_items позицию с order_id нового заказа, product_id ноутбука, quantity = 1, price = 300000.
- Добавьте вторую позицию с тем же order_id, product_id мыши, quantity = 2, price = 8000.
- Проверьте, что обе позиции относятся к одному заказу.

При вводе значений через интерфейс pgAdmin в этих упражнениях сохранение заказа и позиций может происходить отдельными операциями. Это демонстрация структуры данных, а не атомарное оформление заказа. Транзакционное оформление будет изучаться отдельно.

10. Проверка рассчитанной суммы

В Query Tool выполните запрос для проверки сумм по каждому заказу:

```
SELECT order_id, SUM(quantity * price) AS calculated_total
FROM order_items
GROUP BY order_id
ORDER BY order_id;
```

Для тестового заказа результат должен быть 316000: 1 × 300000 + 2 × 8000 = 316000. Сравните его с orders.total_amount. PostgreSQL не обеспечивает их равенство автоматически только за счёт PRIMARY KEY, FOREIGN KEY и CHECK.

11. Проверка ограничений целостности

Выполните тесты на ошибочные данные. Отрицательные проверки удобнее проводить через Query Tool. Не изменяйте и не удаляйте реальные записи; используйте несуществующие идентификаторы и отдельные учебные вставки.

- UNIQUE: попробуйте вставить второго пользователя с email, уже существующим в users. Ожидается ошибка уникальности.

- FOREIGN KEY: попробуйте добавить заказ с user_id, которого нет в users. Ожидается ошибка внешнего ключа.
- FOREIGN KEY: попробуйте добавить позицию с product_id, которого нет в products. Ожидается ошибка внешнего ключа.
- CHECK: попробуйте сохранить quantity = 0. Ожидается ошибка CHECK.
- CHECK: попробуйте сохранить отрицательную цену товара или позиции заказа. Ожидается ошибка CHECK.
- NOT NULL: попробуйте сохранить покупателя без обязательного name. Ожидается ошибка NOT NULL.

Если выполняете ошибки через Query Tool, запускайте каждый запрос отдельно. В случае ошибки внутри явно начатой транзакции завершите её ROLLBACK, прежде чем продолжать.

12. Проверка данных SQL-запросами

Получите содержимое созданных таблиц и убедитесь, что ссылки существуют:

```
SELECT * FROM users ORDER BY id;
SELECT * FROM orders ORDER BY id;
SELECT * FROM order_items ORDER BY id;
```

Для просмотра ограничений и структуры пользуйтесь вкладками Columns/Constraints в Properties каждой таблицы. Повторно откройте pgAdmin и убедитесь, что записи сохранились.

13. Дополнительное задание — самостоятельно

Задание повышенной сложности: «История заказов покупателя». Готовый SQL-запрос не приводится. Необходимо разработать один SELECT, связывающий users, orders, order_items и products посредством JOIN.

- По идентификатору пользователя получить заказы и входящие в них товары.
- Показать идентификатор заказа, имя покупателя, название товара, количество и цену на момент заказа.
- Добавить вычисляемый столбец «Сумма позиции» = quantity × order_items.price.
- Отсортировать результат по номеру заказа и названию товара.
- Не использовать цену из products вместо order_items.price для расчёта стоимости ранее оформленной позиции.
- Проверить результат на заказе с ноутбуком и двумя мышами.

Заказ	Покупатель	Товар	Количество	Цена, ₸	Сумма позиции, ₸
номер заказа	имя покупателя	Ноутбук	1	300000	300000
тот же номер	тот же покупатель	Мышь	2	8000	16000

Дополнительно по желанию: расширьте запрос так, чтобы он показывал общую сумму каждого заказа. Используйте SQL-агрегацию корректно и не допускайте повторного суммирования одних и тех же строк.

14. Что приложить к отчёту

- Скриншоты структуры всех пяти таблиц (Columns).
- Скриншоты всех четырёх внешних ключей (Constraints → Foreign Key).

- Скриншоты и пояснения ограничений UNIQUE и CHECK.
- Результаты выборок users, orders и order_items.
- Расчёт итоговой стоимости тестового заказа.
- Не менее трёх примеров ожидаемо отклонённых некорректных записей с пояснениями.
- При выполнении дополнительного задания — собственный SQL JOIN и его результат.

15. Критерии выполнения

- Существующая БД shop_db и данные products/categories сохранены.
- Созданы таблицы users, orders, order_items с правильными типами и ограничениями.
- Настроены четыре внешних ключа и проверены ссылки на существующие строки.
- Тестовый заказ имеет две позиции; его сумма равна 316000 ₸.
- Продемонстрированы ошибки UNIQUE, FOREIGN KEY, CHECK и NOT NULL.
- Студент объясняет назначение каждой таблицы, PK и FK, а также отличие корзины от оформленного заказа.

16. Контрольные вопросы

- Почему категория хранится отдельно от товара?
- Чем PRIMARY KEY отличается от UNIQUE?
- Где находится внешний ключ связи users → orders и почему?
- Для чего нужна таблица order_items, если есть orders?
- Почему order_items.price нужно сохранять отдельно от products.price?
- Что предотвращает CHECK (quantity > 0)?
- Почему общий итог заказа не вычисляется автоматически одним лишь FOREIGN KEY?
- Почему нельзя доверять цене, присланной браузером, при оформлении заказа?
- Какой недостаток имеет последовательное создание заказа и его позиций без общей транзакции?
- Какие таблицы нужно соединить, чтобы показать историю покупок пользователя?